

Container vs. Virtuelle Maschinen

Ein tiefgehender Lehrgang

Systemsicherheits-Lehrgang

6. Mai 2026

Hervorragend strukturierter Prompt – wir gehen ihn systematisch durch. Ich werde an Stellen, wo es das Verständnis fördert, technisch tief gehen und Beispiele aus der Praxis einstreuen.

Inhaltsverzeichnis

1	Fundamentale Konzepte	1
1.1	Grundprinzipien von Isolation	1
1.2	Container-Technologien	1
1.3	Virtuelle Maschinen	2
1.4	Vergleich Container vs. VM	3
2	Bedrohungsmodelle und Vertrauensgrenzen	3
2.1	Angreiferannahmen	3
2.2	Escape-Szenarien	4
2.3	Kernel als Trusted Computing Base	4
3	Praktische Umsetzung	4
3.1	Einfacher Webserver – Container-Variante	4
3.2	Webserver in der VM	5
3.3	Analyse der Isolation	5
3.4	Gezielte Fehlkonfigurationen (Container)	6
3.5	Gezielte Fehlkonfigurationen (VM)	6
3.6	Warum diese Konfigurationen die Sicherheit schwächen	7
4	Failure- und Escape-Szenarien	7
4.1	Container-Breakout durch Kernel-Bug	7
4.2	Capability-Missbrauch	7
4.3	Hypervisor-Escape	7
4.4	Side-Channel-Angriffe	7
4.5	Vergleich	8
5	Best Practices	8
6	Praxisteil: VM auf macOS	8
6.1	Empfehlung nach Mac-Typ	8
6.2	Setup mit UTM (Apple Silicon)	9
6.3	Worauf du achten solltest	9
	Empfohlene Lernreihenfolge	9

1 Fundamentale Konzepte

1.1 Grundprinzipien von Isolation

Isolation auf Betriebssystemebene bedeutet, dass Prozesse, Ressourcen oder Systeme so voneinander getrennt sind, dass sie sich gegenseitig weder beobachten noch beeinflussen können – außer über klar definierte Schnittstellen. Isolation ist dabei kein binärer Zustand, sondern ein Spektrum mit unterschiedlichen Stärkegraden.

Prozess-Isolation ist die schwächste Form. Der Kernel weist jedem Prozess einen eigenen virtuellen Adressraum zu (über die MMU – Memory Management Unit). Prozess A kann den Speicher von Prozess B nicht direkt lesen. Aber: Beide Prozesse teilen sich denselben Kernel, dasselbe Dateisystem, dieselbe Prozessliste. Ein Prozess kann andere via `ps` sehen, auf gemeinsame Dateien zugreifen, dieselben Systemcalls verwenden.

Kernel-Isolation geht einen Schritt weiter. Hier wird der Userspace zwar isoliert, aber der **Kernel bleibt geteilt**. **Container fallen in diese Kategorie**: Sie sehen voneinander unabhängige Userspaces (eigene Prozessliste, eigenes Filesystem-View, eigene Netzwerk-Interfaces), nutzen aber denselben Linux-Kernel. Die Isolation wird durch Kernel-Features (Namespaces, cgroups) durchgesetzt.

Hardware-Isolation ist die stärkste Form. Hier wird auf CPU-Ebene zwischen verschiedenen „Welten“ unterschieden – typischerweise mittels Hardware-Virtualisierungsfunktionen (Intel VT-x, AMD-V, ARM Virtualization Extensions). Jeder Gast hat seinen eigenen Kernel; der Hypervisor selbst läuft auf einer privilegierten Ebene (oft „Ring -1“ oder „EL2“ auf ARM). **Ein Bug im Gast-Kernel kompromittiert nicht den Host**.

Die Faustregel: **Je tiefer die Isolationsgrenze in der Hardware verankert ist, desto schwerer ist sie zu durchbrechen – und desto höher der Overhead.**

1.2 Container-Technologien

Container sind technisch gesehen **keine eigene Technologie**, sondern eine clevere Kombination bestehender Linux-Kernel-Features. Sie sind im Wesentlichen **„isolierte Prozesse“** – nicht mehr, nicht weniger.

Namespaces sind das primäre Isolationsprimitiv. Ein Namespace ist eine Ressourcen-Sicht, die für eine Gruppe von Prozessen unabhängig ist:

- **PID** – Prozess-IDs. Im Container sieht der Init-Prozess die PID 1, obwohl er auf dem Host vielleicht PID 23456 ist. Prozesse außerhalb des Namespaces sind unsichtbar.
- **NET** – Netzwerkstack. Eigene Interfaces, Routing-Tabellen, iptables-Regeln, Ports.
- **MNT** – Mount-Points. Das Filesystem-Layout wirkt wie ein eigenständiges System (über `pivot_root` oder `chroot`).
- **IPC** – Inter-Process Communication (System-V-IPC, POSIX-Message-Queues). Verhindert, dass Container über Shared Memory miteinander reden.
- **UTS** – Hostname und Domainname.
- **USER** – User-IDs. Hier liegt eine wichtige Sicherheitsfunktion: Ein User mit UID 0 (root) im Container kann auf eine unprivilegierte UID auf dem Host gemappt werden (User Namespace Remapping).
- **CGROUP** – Sicht auf cgroup-Hierarchien.
- **TIME** – (neuer) eigene monotonic/boottime-Zeit.

Control Groups (cgroups) sind orthogonal zu Namespaces. Während Namespaces *Sichtbarkeit* einschränken, beschränken cgroups *Ressourcenverbrauch*: CPU-Anteile, Memory-Limits, Block-I/O-Bandbreite, Anzahl von PIDs. Ohne cgroups könnte ein Container per Fork-Bombe das Host-System zum Stillstand bringen.

Geteilter Kernel – die zentrale Sicherheitsfrage: Alle Container auf einem Host nutzen *denselben Linux-Kernel*. Das hat enorme Vorteile (Effizienz, Startgeschwindigkeit, geringer Overhead), aber eine kritische Konsequenz: **Jede Kernel-Schwachstelle ist potenziell ein Container-Escape**. Der Kernel ist die einzige Sicherheitsgrenze. Wenn ein Container einen Kernel-Bug ausnutzt (z.B. Privilege Escalation in einem Syscall), gewinnt er Zugriff auf den Host und damit auf alle anderen Container.

Zusätzlich kommen weitere Schutzmechanismen ins Spiel, die die Angriffsfläche reduzieren: **Capabilities** (feingranulare Aufteilung der Root-Rechte – Container bekommen normalerweise nur ca. 14 von ca. 40 Capabilities), **Seccomp-BPF** (Whitelist erlaubter Syscalls – Docker default blockt ca. 60 von ca. 330), **AppArmor/SELinux** (Mandatory Access Control auf Pfade/Operationen).

1.3 Virtuelle Maschinen

VMs simulieren komplette Computer. Jede VM hat ihren eigenen Kernel, ihre eigene Hardware-Sicht, ihren eigenen Bootloader.

Hypervisor-Typen:

- **Typ 1 (Bare-Metal):** Läuft direkt auf der Hardware. Beispiele: KVM (auf Linux genaugenommen ein Hybrid), Xen, VMware ESXi, Microsoft Hyper-V. Sehr performant, da kein Host-OS dazwischen liegt.
- **Typ 2 (Hosted):** Läuft als Anwendung auf einem normalen Betriebssystem. Beispiele: VirtualBox, VMware Workstation, Parallels Desktop, UTM (auf macOS via Apple's Virtualization.framework oder QEMU).

Die Grenze ist heute fließend. KVM unter Linux ist technisch ein Kernel-Modul, das aber Hardware-Virtualisierung direkt nutzt – also funktional ein Typ-1-Hypervisor mit Linux als Management-OS.

Hardware-Virtualisierung: Moderne CPUs bieten Erweiterungen, mit denen Gast-Code nahezu nativ ausgeführt werden kann:

- **Intel VT-x / AMD-V:** Führen einen neuen CPU-Modus ein („VMX root“ vs. „VMX non-root“). Privilegierte Operationen im Gast lösen einen „VM Exit“ aus, den der Hypervisor behandelt.
- **EPT/NPT (Extended/Nested Page Tables):** Hardware-Übersetzung von Gast-physikalischen zu Host-physikalischen Adressen. Ohne dies wäre Memory-Virtualisierung extrem teuer.
- **Apple Silicon:** ARM64 mit ARMv8.x Virtualization Extensions. Apple stellt das Virtualization.framework bereit, das Hypervisor-Funktionalität abstrahiert. Wichtig: **Es können nur ARM-Gäste ausgeführt werden** (oder x86-Gäste mit deutlichem Performance-Verlust durch Emulation, z.B. via Rosetta in QEMU).

Virtuelle Hardware: Der Hypervisor präsentiert dem Gast emulierte oder paravirtualisierte Geräte. Reine Emulation (z.B. emulierte Intel-NIC) ist langsam aber kompatibel; **Paravirtualisierung** (virtio unter Linux/KVM) bietet hocheffiziente Geräte, die wissen, dass sie virtuell sind, und über Ringbuffer mit dem Host kommunizieren.

1.4 Vergleich Container vs. VM

Dimension	Container	VM
Isolationsgrenze	Kernel (geteilt)	Hardware (Hypervisor)
Boot-Zeit	Millisekunden	Sekunden bis Minuten
Memory-Overhead	MB	Hunderte MB bis GB
Dichte pro Host	Hunderte	Dutzende
Angriffsfläche	Kompletter Kernel-Syscall-Interface (~330 Syscalls)	Schmales Hypervisor-Interface (Hypercalls + emulierte Geräte)
OS-Vielfalt	Nur gleicher Kernel (Linux-Container brauchen Linux-Host)	Beliebige Gast-OS
Typische Einsätze	Microservices, CI/CD, Stateless Apps	Multi-Tenant Cloud, Legacy-Workloads, OS-Diversität

Eine wichtige Nuance: Performance-Overhead bei modernen VMs ist *für CPU-bound Workloads* minimal (oft < 5 %). Der Hauptkostenfaktor ist Memory (jede VM hat einen vollständigen Kernel + Userspace) und I/O-Latenz.

2 Bedrohungsmodelle und Vertrauensgrenzen

2.1 Angreiferannahmen

Container-Bedrohungsmodell: Standardannahme ist, dass der Angreifer Code im Container ausführen kann (z.B. durch eine kompromittierte Webanwendung, einen böartigen Container-Image, oder einen Insider). Ziel des Angreifers: aus dem Container auszubrechen, andere Container zu beeinflussen oder den Host zu kompromittieren.

Das Vertrauensmodell ist hier wichtig: **Container schützen nicht vor böartigen Workloads, sondern vor versehentlichen Konflikten zwischen vertrauenswürdigen Workloads.** Das ist in der Industrie ein viel diskutierter Punkt – Google und andere argumentieren, dass für echte Multi-Tenancy zusätzliche Layer (gVisor, Kata Containers, Firecracker) nötig sind, die Hardware-Virtualisierung mit Container-UX kombinieren.

VM-Bedrohungsmodell: Standardannahme ist, dass der Gast komplett kompromittiert ist – Angreifer hat Root im Gast-Kernel. Trotzdem soll der Host und andere Gäste sicher bleiben. Das ist ein deutlich stärkeres Modell, weshalb VMs die Standardgrenze für Multi-Tenant-Public-Cloud sind (AWS EC2, Azure VMs).

2.2 Escape-Szenarien

Container-Escapes sind dokumentiert und realistisch:

- **CVE-2019-5736** (runc): Ein böartiges Container-Image konnte den `runc`-Binary auf dem Host überschreiben, indem es auf `/proc/self/exe` zugriff. Bei nächstem Container-Start wurde der Angreifer-Code mit Host-Root ausgeführt.
- **CVE-2022-0492** (cgroups v1): Über die `release_agent`-Funktionalität konnte ein Container mit `CAP_SYS_ADMIN` Code als Host-Root ausführen.
- **Dirty Pipe (CVE-2022-0847)**: Kernel-Bug in der Pipe-Implementierung erlaubte Schreibzugriff auf read-only Dateien – auch aus Containern heraus auf Host-Dateien.

Das Muster ist meist: Angreifer findet einen Kernel-Bug oder eine Fehlkonfiguration und nutzt sie, weil eben **derselbe Kernel** die einzige Trennung darstellt.

VM-Escapes sind seltener, aber spektakulärer:

- **VENOM (CVE-2015-3456)**: Bug im virtuellen Floppy-Controller von QEMU – ein Gast konnte Out-of-Bounds-Writes im Hypervisor-Speicher auslösen.
- **CVE-2017-2636** und diverse VirtualBox-CVEs in der virtuellen Netzwerkkarte.
- **Spectre/Meltdown** und Folge-CVEs: Side-Channel-Angriffe, die theoretisch Daten zwischen VMs lecken können, ohne klassischen Escape.

Die Hypervisor-Angriffsfläche ist deutlich kleiner – ein paar Hypercalls und emulierte Geräte – aber auch hier gilt: jede Code-Zeile, die der Gast erreichen kann, ist ein potenzieller Angriffsvektor.

2.3 Kernel als Trusted Computing Base

Die **TCB** ist die Menge an Code, die korrekt funktionieren *muss*, damit das Sicherheitsmodell hält. Bei Containern ist die TCB der **gesamte Linux-Kernel** – inklusive aller Treiber, des kompletten Syscall-Interfaces, aller Filesystem-Implementierungen. Das sind ca. 30 Millionen Zeilen Code. Bei VMs ist die TCB der **Hypervisor** – z.B. KVM mit etwa ~100.000 Zeilen Code im hot path. Das ist ein Unterschied von zwei bis drei Größenordnungen in der Angriffsfläche.

Genau deshalb haben Lösungen wie **gVisor** (User-Space-Kernel-Reimplementation, die Syscalls abfängt) oder **Kata Containers** (jeder Container in einer Mikro-VM) Existenzberechtigung: Sie verkleinern die TCB drastisch.

3 Praktische Umsetzung

3.1 Einfacher Webserver – Container-Variante

Hier ein minimaler Flask-Server. Erstelle die Dateien in einem Verzeichnis `webapp/`:

`app.py`:

```
from flask import Flask
import os, socket
app = Flask(__name__)

@app.route("/")
def home():
    return f"Hello from {socket.gethostname()}, PID {os.getpid()}\n"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
RUN pip install flask
USER 1000:1000
EXPOSE 5000
CMD ["python", "app.py"]
```

Bauen und starten:

```
docker build -t flask-demo .
docker run -d --name flask-secure -p 5000:5000 flask-demo
curl http://localhost:5000
```

Beachte das USER 1000:1000 – das ist wichtig: Standardmäßig laufen Container als Root, was die Angriffsfläche unnötig vergrößert.

3.2 Webserver in der VM

In einer Ubuntu-VM (siehe macOS-Setup unten):

```
sudo apt update && sudo apt install -y python3-pip
pip3 install flask --break-system-packages
# app.py wie oben
python3 app.py
```

Vom Host aus erreichbar über die VM-IP (bei NAT braucht es Port-Forwarding, bei Bridged direkt über die DHCP-zugewiesene IP).

3.3 Analyse der Isolation

Im Container ausführen:

```
docker exec flask-secure sh -c "ps -ef; ls /; mount | head; cat /proc/1/status | grep CapEff"
```

Beobachte: PID 1 ist dein Python-Prozess (PID-Namespace), das Filesystem ist die Image-View (Mount-Namespace), Capabilities sind reduziert.

In der VM hingegen:

```
ps -ef          # vollstaendiger Prozessbaum inkl. systemd, kthreads
mount           # vollstaendige Mount-Tabelle, eigene Block-Devices
uname -r        # eigener Kernel!
```

Das ist der fundamentale Unterschied: In der VM siehst du einen vollständigen Kernel mit eigenen Kernel-Threads ([kthreadd], [ksoftirqd] etc.). Im Container existieren diese gar nicht – der Container *hat* keinen eigenen Kernel.

Syscall-Vergleich:

```
# Im Container
docker exec flask-secure sh -c "apt update && apt install -y strace 2>/dev/null; \
  strace -c -f -p 1 & sleep 2; kill %1"
```

Dadurch siehst du, welche Syscalls der Container überhaupt nutzt – meist eine kleine Teilmenge.

3.4 Gezielte Fehlkonfigurationen (Container)

-privileged: Hebelt nahezu alle Schutzmechanismen aus.

```
docker run -d --name flask-priv --privileged -p 5001:5000 flask-demo
docker exec flask-priv sh -c "cat /proc/1/status | grep CapEff"
# CapEff: 000001ffffffff -- alle Capabilities!
```

Im privileged-Container kannst du Kernel-Module laden, das Host-Filesystem mounten, auf Host-Devices zugreifen. Das ist effektiv kein Container mehr.

Docker-Socket mounten:

```
docker run -d --name flask-sock -v /var/run/docker.sock:/var/run/docker.sock flask-demo
```

Das ist ein klassischer Anti-Pattern. Der Container kann jetzt selbst die Docker-API ansprechen und z.B. einen privilegierten Container starten, der das Host-Filesystem mountet. Voller Host-Zugriff.

Seccomp deaktivieren:

```
docker run -d --security-opt seccomp=unconfined flask-demo
```

Jetzt sind alle ca. 330 Syscalls verfügbar – auch obskure wie `keyctl`, `userfaultfd`, `bpf` und Ähnliches, die häufig in Kernel-Exploits genutzt werden.

Shared Volumes mit Host-Pfaden:

```
docker run -v /:/host flask-demo # KATASTROPHE
```

Container kann das gesamte Host-Dateisystem lesen und schreiben.

3.5 Gezielte Fehlkonfigurationen (VM)

- **Bridged Networking:** Die VM erscheint als eigenständiges Gerät im Host-Netzwerk. Aus Bequemlichkeit oft genutzt, aber: Wenn die VM kompromittiert ist, kann sie ARP-Spoofing, DHCP-Snooping und Lateral Movement im LAN versuchen.
- **Shared Clipboard / Drag-and-Drop:** Implementiert über Gast-Tools (z. B. `open-vm-tools`, `Parallels Tools`). Das ist zusätzlicher privilegierter Code im Gast und ein zusätzliches Hypervisor-Interface – also Angriffsfläche.
- **Shared Folders (z. B. via `virtfs/9p`):** Macht Host-Dateien direkt erreichbar. Bei Bugs im 9p-Server (es gab welche) wird das zur Escape-Vektor.
- **Nested Virtualization aktiviert ohne Notwendigkeit:** Erweitert das Hypervisor-Interface.

3.6 Warum diese Konfigurationen die Sicherheit schwächen

Das Muster ist immer dasselbe: **Jede Lockerung der Isolation erweitert die Angriffsfläche.** Wenn du den Docker-Socket exposest, ist der Schutz des Containers irrelevant – der Angreifer geht „drumrum“ über die Container-Engine. Wenn du `-privileged` setzt, gibt es schlicht keine Sicherheitsgrenze mehr. Bei VMs sind die Auswirkungen subtiler, weil die Hypervisor-Grenze meist hält, aber jeder zusätzliche Kommunikationskanal ist ein potenzieller Bug-Vektor.

4 Failure- und Escape-Szenarien

4.1 Container-Breakout durch Kernel-Bug

Konkretes Beispiel **Dirty Pipe (CVE-2022-0847)**: Ein Bug in `copy_page_to_iter_pipe()` erlaubte das Schreiben in Page-Cache-Seiten, die zu read-only-geöffneten Dateien gehörten – sogar zu Files mit eingeschränkten Permissions. Aus einem Container heraus konnte man so z.B. das `/etc/passwd` des Hosts modifizieren (wenn es überlappende Pages im Cache gab) oder, eleganter, in die Binary von Setuid-Programmen schreiben.

Der Punkt: Der Container hat keinen Bug ausgenutzt – der *Kernel* hatte den Bug, und der Container nutzt denselben Kernel.

4.2 Capability-Missbrauch

`CAP_SYS_ADMIN` ist die „neue Root“ – wer sie hat, kann fast alles. Beispiel: Mit `CAP_SYS_ADMIN` kannst du `unshare`, neue Mount-Namespaces erstellen, und über die historische `release_agent`-Funktion in cgroups v1 Code als Host-Root ausführen lassen.

`CAP_SYS_MODULE` erlaubt das Laden von Kernel-Modulen – sofortige Host-Kompromittierung, da das Modul im Kernel-Kontext läuft.

`CAP_DAC_READ_SEARCH` umgeht Datei-Permission-Checks beim Lesen. Mit `open_by_handle_at()` kann man so beliebige Files lesen, sogar solche, die im Container-Namespace gar nicht sichtbar sind (klassische „Shocker“-Exploit-Technik).

4.3 Hypervisor-Escape

VENOM (CVE-2015-3456): Ein Buffer Overflow im virtuellen Floppy Disk Controller (FDC) von QEMU. Selbst wenn die Floppy in der VM-Konfiguration deaktiviert war, war der FDC-Code im Hypervisor-Prozess geladen und ansprechbar. Ein Gast konnte durch spezielle FDC-Befehle den Heap des QEMU-Prozesses überschreiben und Code-Ausführung im Host erreichen. Jahrelang offen, weil nostalgischer Legacy-Code selten gepatcht wird.

Allgemeines Muster: Hypervisor-Escapes finden meist in emulierten Geräten statt (Netzwerk, Storage, Grafik, USB), seltener im Hypercall-Interface. Lehre: **Geräte deaktivieren, die du nicht brauchst.**

4.4 Side-Channel-Angriffe

Spectre/Meltdown und Nachfolger (Foreshadow, MDS, Zombieload, Retbleed) nutzen mikroarchitekturelle Effekte – Cache-Timing, Branch Prediction, spekulative Ausführung – um Daten zu lecken, die das Programm „sehen können dürfte, aber nicht direkt zurückgeben darf“.

Im VM-Kontext bedeutet das: Ein Gast kann unter Umständen Daten anderer Gäste oder des Hypervisors lecken – ohne klassischen Escape. Mitigations (KPTI, Retpoline, IBRS, etc.) kosten Performance, sind aber Standard. AWS und Azure haben hier massiv investiert; selbst gehostete VMs benötigen disziplinierte Patch-Hygiene.

4.5 Vergleich

Aspekt	Container-Escape	Hypervisor-Escape
Eintrittswahrscheinlichkeit	Hoch (große Angriffsfläche)	Niedrig (kleine Angriffsfläche)
Komplexität des Angriffs	Mittel (oft existieren PoCs für Kernel-CVEs)	Hoch (Hypervisor-Internals, oft Heap-Spraying)
Schadensausmaß	Vollständige Kompromittierung	Vollständige Kompromittierung, aber mehr Tenants betroffen
Patch-Kadenz	Linux-Kernel-Patches monatlich, oft kritisch	Hypervisor-Patches seltener, aber auch seltener nötig

5 Best Practices

Sichere Container-Konfiguration: Niemals als Root laufen (`USER` im Dockerfile). Capabilities reduzieren (`-cap-drop=ALL -cap-add=NET_BIND_SERVICE` etc.). Read-only Filesystem (`-read-only`). Eigene Seccomp-Profil, die nur tatsächlich benötigte Syscalls erlauben. AppArmor/SELinux

nicht deaktivieren. **Niemals -privileged.** Niemals den Docker-Socket reichen. User-Namespace-Remapping aktivieren, sodass Container-Root nicht Host-Root ist. Images von vertrauenswürdigen Registries beziehen und mit Tools wie Trivy oder Gype scannen. Image-Signing (Cosign, Notary) nutzen.

VM-Härtung: Nicht benötigte virtuelle Geräte entfernen (USB-Controller, Audio, Floppy!), serielle Schnittstellen). Gast-Tools nur installieren, wenn wirklich nötig – sie sind Komfort, aber Angriffsfläche. Snapshots vor Experimenten. Memory Ballooning und Page Sharing kritisch betrachten (KSM hatte schon Probleme). Hypervisor regelmäßig patchen.

Netzwerksegmentierung: Container in eigene Netze (Docker User-Defined Networks, Kubernetes Network Policies). VMs in VLANs. Default-Deny im Egress – ausgehender Verkehr ist für Lateral Movement und C2 entscheidend. Im Cloud-Kontext: Security Groups so eng wie möglich.

Least Privilege: Auf jeder Ebene. Container-User, VM-User, Cloud-IAM-Rollen, Service Accounts. Wenn du dich fragst „brauche ich diese Berechtigung?“ – wahrscheinlich nicht.

Monitoring und Logging: Auditd auf dem Host, Falco oder Tracee für Container-Runtime-Anomalien (z. B. unerwartete Syscalls, Schreibzugriffe in /etc, Spawn von sh in Web-Containern). Hypervisor-Logs sammeln. Anomalien automatisiert melden – nicht nur loggen.

6 Praxisteil: VM auf macOS

6.1 Empfehlung nach Mac-Typ

Auf **Apple Silicon** (M1/M2/M3/M4) ist **UTM** der pragmatische Standard – kostenlos, Open Source, basiert auf QEMU mit Apple's Virtualization.framework als optionalem Backend. Das Virtualization.framework ist deutlich schneller als reine QEMU-Emulation, unterstützt aber nur ARM-Linux-Gäste. **Parallels Desktop** ist sehr glatt, aber kostenpflichtig. **VirtualBox** auf Apple Silicon: lange Zeit unzureichend, mittlerweile als ARM64-Build verfügbar, aber mit Einschränkungen.

Auf **Intel-Macs** funktionieren VirtualBox, VMware Fusion (mittlerweile gratis für Personal Use) und Parallels.

6.2 Setup mit UTM (Apple Silicon)

1. UTM von utm.app installieren
2. Ubuntu Server ARM64 herunterladen (Ubuntu 24.04 LTS, `arm64.iso`)
3. In UTM „Create a New Virtual Machine“ → „Virtualize“ (nicht „Emulate“ – Letzteres wäre für x86-Gäste, deutlich langsamer) → Linux
4. ISO auswählen, RAM ≥ 4 GB, CPU 2–4 Kerne, Storage 20 GB
5. Netzwerk: Standard ist „Shared Network“ (NAT) – sicherer Default. Für Realismus später auf „Bridged“ umstellen, dann erscheint die VM als Gerät im LAN.
6. Nach der Installation: `sudo apt update && sudo apt install -y openssh-server docker.io` – damit hast du eine Linux-VM, in der du wiederum Container starten kannst.

6.3 Worauf du achten solltest

Architektur: Apple Silicon ist ARM64. Verwende **immer** ARM-Images (Ubuntu, Debian, Alpine bieten alle native ARM64-Varianten). x86-Images via Emulation funktionieren, sind aber 5–10x langsamer.

Netzwerkmodi:

- *NAT/Shared*: VM bekommt private IP, Host macht NAT. Gut isoliert, aber von außen nicht erreichbar (außer Port-Forwarding).
- *Bridged*: VM erscheint als eigenes Gerät im LAN. Realistischer, riskanter.
- *Host-only*: Nur Host und VM können kommunizieren. Ideal für Experimente.

Komfort-Features kritisch betrachten: Shared Clipboard, Shared Folders, Drag-and-Drop sind enorm bequem, aber jedes davon ist ein zusätzlicher Code-Pfad zwischen Gast und Host. Für reine Sicherheits-Experimente: aus.

Snapshots: Vor jedem riskanten Experiment. UTM, Parallels und VirtualBox unterstützen das alle. Damit kannst du eine „kompromittierte“ VM in Sekunden zurücksetzen.

Container in VM: Sehr empfehlenswert – installiere Docker oder Podman in der Linux-VM und führe die Container-Experimente *dort* aus. Das gibt dir eine zusätzliche Isolationsebene zwischen Experiment und macOS-Host und eine realistische Linux-Umgebung (macOS-Docker läuft *immer* in einer VM, intransparenter).

Empfohlene Lernreihenfolge

Konkret vorgeschlagen:

1. **Theorie verinnerlichen** – Lies Teile 1–2 nochmal, mache dir Notizen zu Namespaces und Capabilities. Schau dir `man 7 namespaces` und `man 7 capabilities` auf einem Linux-System an.
2. **VM einrichten** – Ubuntu in UTM, mit Snapshot nach der Grundinstallation.
3. **Parallel-Setup** – Docker in der VM, Flask-App sowohl als Container als auch direkt.
4. **Bewusst unsicher** – Spiel die Fehlkonfigurationen aus Teil 3.4/3.5 durch und beobachte mit `nsenter`, `lsns`, `capsh -print`, was sich ändert.
5. **Beobachten** – Was siehst du im Container, das du im Host beeinflussen kannst? Wo hört die Sicht auf?
6. **Dokumentieren** – Schreib für jeden Versuch auf: Was habe ich gemacht, was war die Hypothese, was war das Ergebnis, welche Sicherheits-Implikation.

Wenn du mit einem konkreten Schritt loslegen willst – etwa dem ersten Container-Escape-Experiment in einer kontrollierten Umgebung, oder einer detaillierten Walkthrough des **-privileged-**Missbrauchs – sag Bescheid, dann gehen wir das gezielt durch.